

```
int res;
char i;
void main() {
for (i=0;i<5;i++){
    res = arrpf[i](10,20);
}
} //~!
```

4.2.5 Structures

A structure is a derived type usually representing a user-defined collection of named members (or components). The members can be of any type, either fundamental or derived (with some restrictions to be noted later), in any sequence. In addition, a structure member can be a bit field type not allowed elsewhere.

Unlike arrays, structures are considered to be single objects. The CCS structure type lets you handle complex data structures almost as easily as single variables. CCS does not support anonymous structures (ANSI divergence).

Structure declaration and initialisation

Structures are declared using the keyword struct:

```
structtag {member-declarator-list};
```

Here, tag is the name of the structure; member-declarator-list is a list of structure members, actually a list of variable declarations. Variables of structured type are declared same as variables of any other type.

The member type cannot be the same as the struct type being currently declared. However, a member can be a pointer to the structure being declared, as in the following example:

```
structmystruct {mystruct s;}; /* illegal! */
structmystruct {mystruct *ps;}; /* OK */
```

Also, a structure can contain previously defined structure types when declaring an instance of a declared structure. Here is an example:

```
/* Structure defining a dot: */
struct Dot {float x, y;};
/* Structure defining a circle: */
struct Circle {
float r;
struct Dot center;
```